Connecting Aspirations

PROJECT REPORT

ON

TORQUE MAP GENERATION

An ML-Based Torque Prediction System for Virtual Electric Vehicle

Mentor: Samarth Surlekar
Student Name: Atharva Dilip Bhosale
Department: Artificial Intelligence and Machine Learning (AIML)
College: ISBM College of Engineering, Pune
Academic Year: 2026 - 2027

Abstract

This report presents the design and implementation of an ML-based torque prediction system for an electric vehicle (EV). The objective of the project is to predict VCU_DesiredTorque from high-frequency (100 Hz) CAN bus data collected during driving trials, spanning hundreds of signals across the inverter, BMS, VCU, OBC, DCDC, and drivetrain systems. Separate models were trained per drive mode — Eco, Sports, and Regen — with Sports mode further split at 3000 RPM into low-speed and high-speed regimes to account for the differing torque-speed relationship in each region.

Exploratory data analysis revealed abnormal signal points and unexpected torque behaviour at zero pedal position, which were investigated, verified against related physical signals, and cleaned prior to model development. Regime-specific models (Random Forest Regressor for the Sports low- and high-speed regimes, XGBoost for Eco and Regen) were developed, with lag features added to the low-speed Sports model to give it temporal memory, improving RMSE.

Post-processing techniques including Exponential Moving Average (EMA) smoothing, gradient/value clipping, and physical torque clamping were applied to suppress prediction spikes observed under aggressive driver behaviour, in preference to a parametric exponential torque-ramp approach that was found to reduce representativeness of actual vehicle behaviour. For Regen coasting, an XGBoost regressor was combined with a parametric transient ramp model to smoothly handle the first samples of a regen event before handing off to the tree-based model.

Finally, the trained Python models were converted to static C headers and integrated into a self-contained C99 implementation (torque_predictor.c) deployed inside a MATLAB Simulink C Caller Block, enabling the prediction pipeline to run without a Python runtime dependency in the vehicle's control software.

1. Introduction

Electric vehicles (EVs) rely on the Vehicle Control Unit (VCU) to compute a desired torque command (VCU_DesiredTorque) from driver inputs and vehicle state, which is then dispatched to the motor controller. Building an accurate, generalizable torque prediction model from real driving data is useful both for validating the existing control logic and for exploring data-driven torque estimation as a complementary or supporting mechanism within the control software.

This project, Torque Map Generation, develops such a system using CAN bus data logged during EV driving trials. The data is high frequency (100 Hz), spread across multiple CSV files, and contains hundreds of signals from the inverter, Battery Management System (BMS), VCU, On-Board Charger (OBC), DC-DC converter, and drivetrain subsystems. The end goal is a torque prediction system that generalizes across driving conditions and drive modes, and that can ultimately be deployed as embedded C code inside the vehicle's MATLAB/Simulink control software.

2. Problem Statement

Given multi-channel, high-frequency CAN bus signals recorded across different drive modes (Eco, Sports, and Regen) and driving styles, the problem is to build machine learning models that accurately predict VCU_DesiredTorque in real time, while being robust to abnormal sensor readings, mode-dependent torque-speed relationships, and varying driver behaviour (e.g. gradual vs. aggressive pedal transitions). The resulting models must further be converted into a form that can execute deterministically inside an embedded C environment (a MATLAB Simulink C Caller Block) with no Python runtime dependency.

3. Objectives

- To analyse and clean high-frequency CAN bus driving data across Eco, Sports, and Regen drive modes.
- To identify and handle abnormal/anomalous signal points without discarding genuine driver-behaviour patterns.
- To divide the Sports-mode data into physically meaningful regimes (Regen, low-speed, high-speed) and develop regime-specific models.
- To engineer temporal and interaction features (lags, rolling statistics, rate-of-change, polynomial terms, RPM zone flags) that improve model accuracy.
- To train and evaluate per-mode torque prediction models (Eco, Sports low/high-speed, Regen) and quantify their accuracy using MAE, RMSE, and R^2.
- To design a post-processing pipeline (clamping, boosting, spike gating, EMA smoothing, rate-limiting) that produces smooth, physically plausible torque output.

- To convert the trained Python models into static C headers and integrate them into a self-contained C99 implementation deployable inside a MATLAB Simulink C Caller Block.

4. Project Overview

The project is organized around three drive-mode-specific model development streams (Eco, Sports, Regen) that feed into a single production integration folder containing the final embedded C deliverable. Mode selection at runtime is governed by the Power_ECO_and_Regen_Mode signal:

Mode Code	Drive Mode	Behaviour
0	Neutral	Returns 0 Nm torque at all times
1	Sports	Split internally at 3000 RPM into two separate models
2	Eco	Single model covering the full torque > 0 region
3	Regen	XGBoost model combined with a parametric transient ramp

Table 1: Drive mode dispatch used by the torque predictor.

The repository is structured into four main areas: ECO/, Regenn/, and Sports/ hold the full model-development lifecycle (data, EDA, training, testing) for each drive mode respectively, while Torque_pred_with_regen is the final production folder containing the deployable C predictor, generated headers, test harness, and validation plots.

5. System Architecture

The deployed system is implemented as torque_predictor.c, a self-contained C99 module that runs inside a MATLAB Simulink C Caller Block. It exposes an initialization function, a per-tick step function, a reset function, and a small diagnostics API:

- TorquePredictor_InitOnce() — one-time initialization, guarded against Simulink's multi-initialization behaviour using a global_initialized flag.
- TorquePredictor_Step() — the per-CAN-tick prediction entry point; takes six inputs, including VCU_MinTorqueLimit and BMS_SOC (both required by the Regen model's feature set), performs mode dispatch, feature engineering, model inference, and post-processing.
- TorquePredictor_Reset() — resets internal history buffers, mirroring dataset segment boundaries used during training/testing.

- TorquePredictor_GetCallCount() / TorquePredictor_GetLastInputs() — read-only diagnostics used to catch integration issues such as call-rate mismatches against the real vehicle sample time.

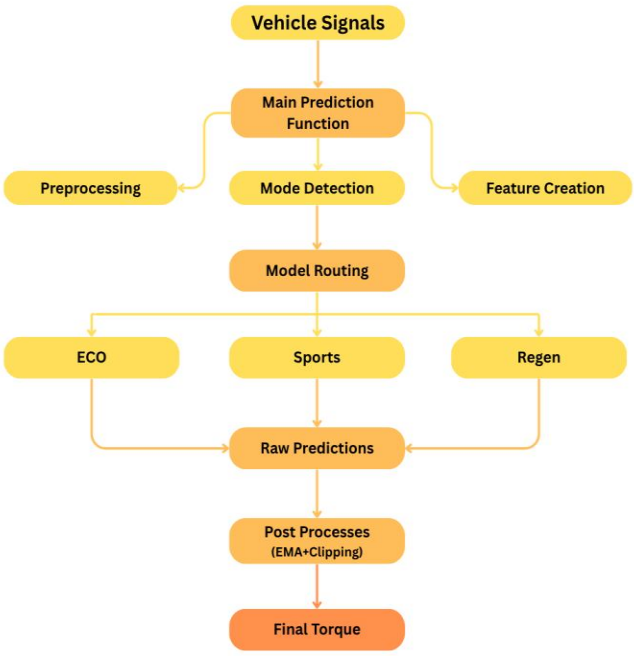


Figure 1: Overall prediction workflow — the predictor() function loading and dispatching across the Eco, Sports, and Regen models.

6. Dataset

The dataset consists of CAN bus logs recorded during EV driving trials at a sampling frequency of 100 Hz, split across multiple CSV files and hundreds of signals spanning the inverter, BMS, VCU, OBC, DC-DC converter, and drivetrain systems. Data was collected separately for each drive mode, identified by mode code: Sports-mode samples (mode code 1), Eco-mode samples (mode code 2), and Regen-mode samples (mode code 3).

Additional batches of processed log data were incorporated over the course of the project, including a third batch of log data used for further training and validation, and a dedicated sports dataset (also sampled at 100 Hz) used specifically to test model performance against real, more aggressive driver behaviour.

The primary prediction target throughout is VCU_DesiredTorque.

7. Exploratory Data Analysis

Exploratory data analysis was carried out on the Sports-mode data to understand signal behaviour and identify data-quality issues before model development.

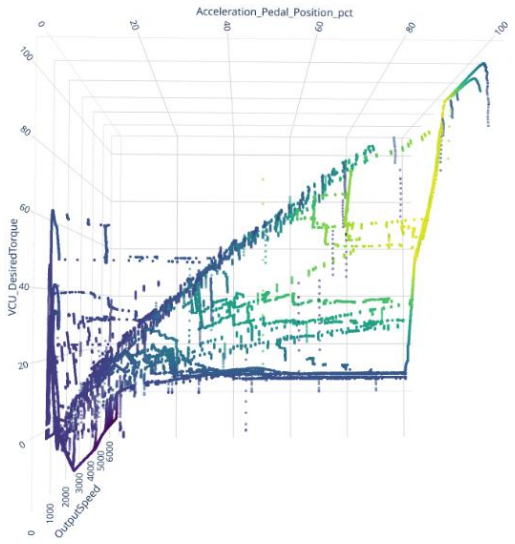


Figure 2: Exploratory analysis of Sports-mode signals showing the overall data distribution.

Two key insights emerged from this analysis:

- Thread-like abnormal points were observed for OutputSpeed values near 100.
- VCU_DesiredTorque increased unexpectedly even when the pedal position was zero, while OutputSpeed remained relatively low for the requested torque.

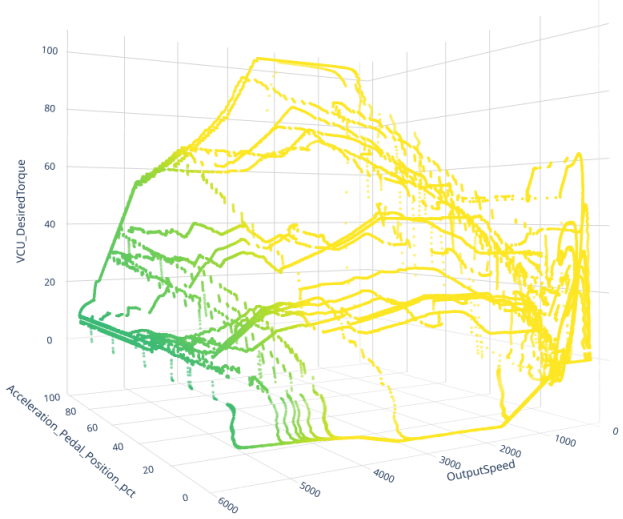


Figure 3: VCU_DesiredTorque rising unexpectedly at zero pedal position while OutputSpeed remained low.

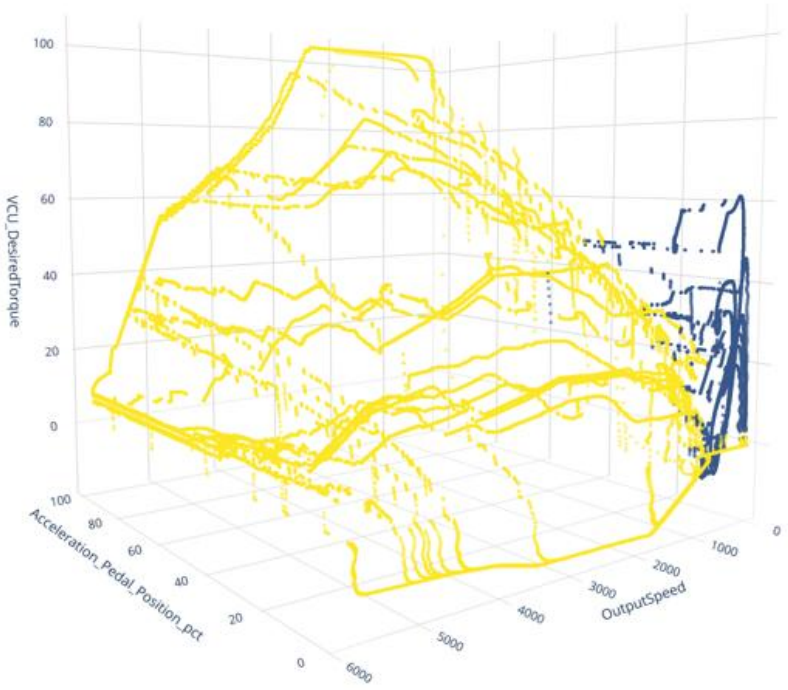


Figure 4: Close-up view of the abnormal signal region identified during EDA.

8. Data Preprocessing

8.1 Removal of Abnormal Points

Following the EDA insights, the identified abnormal points (unexpected torque values inconsistent with pedal position and OutputSpeed) were removed from the training data after verification against related physical signals, to avoid treating genuine driver-behaviour patterns as noise.

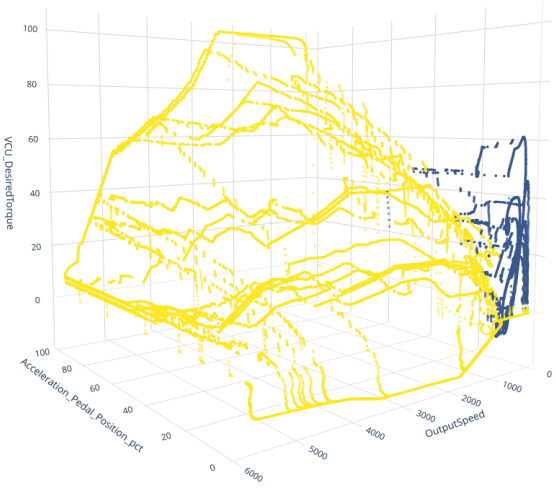


Figure 5: Data distribution of abnormal points.

8.2 Regime Creation

After cleaning, the Sports-mode data was divided into three physically meaningful regimes, since the torque-speed relationship differs significantly between them:

- Regen regime: VCU_DesiredTorque < 0
- Low-speed regime: 100 < OutputSpeed ≤ 3000 RPM
- High-speed regime: OutputSpeed > 3000 RPM

9. Feature Engineering

Two categories of feature engineering were used across the project: general temporal/interaction features embedded in the deployed C predictor, and a specific 8-feature set used for the Regen coasting model.

9.1 Features Used in the Deployed Predictor

The embedded torque_predictor.c feature pipeline replicates the Python training pipelines and includes lag features (up to 4 steps), rolling means, rate-of-change terms, polynomial/interaction terms, and RPM zone flags, computed via warm-started static history buffers so that predictions remain consistent from the first CAN tick onward.

9.2 Lag Features for the Sports Low-Speed Model

For the Sports low-speed regime specifically, lag features were added to give the Random Forest model temporal memory, which substantially reduced residuals (see Section 11).

9.3 Regen Coasting Feature Set

#	FeatureS
1	OutputSpeed
2	VCU_MinTorqueLimit
3	speed_lag1
4	speed_rate
5	torque_limit_lag1
6	torque_lag1
7	soc_lag1
8	samples_since_event_start

Table 2: Input feature set for the Regen coasting XGBoost model.

10. Model Development

10.1 Sports Mode

Following regime creation, a distinct model-selection strategy was used for each Sports-mode regime.

Regen regime (within Sports mode)

For the regen regime observed in Sports mode, an extracted curve is used to predict the regen data points rather than a trained regression model.

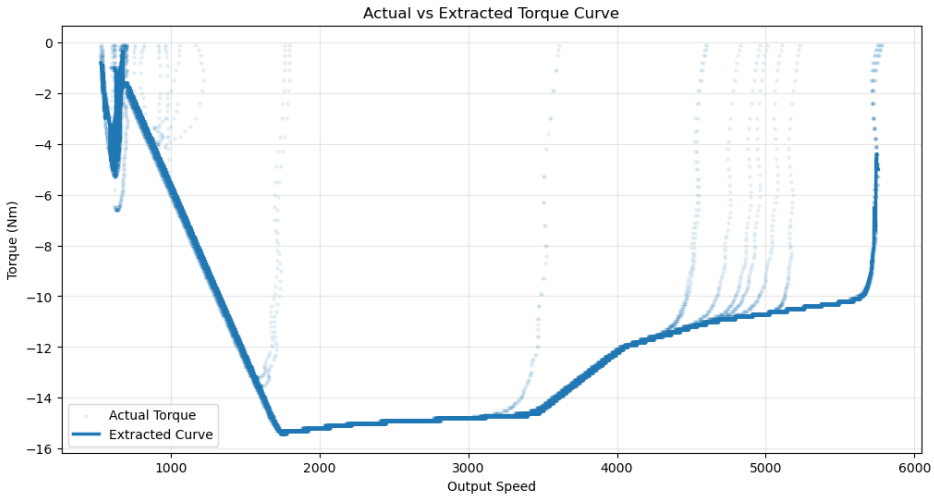


Figure 6: Extracted curve used to predict Sports-mode regen data points.

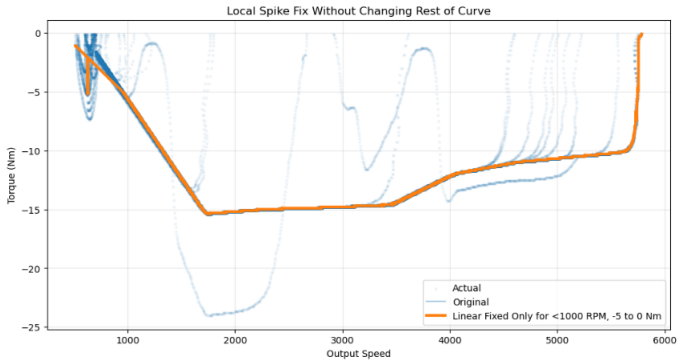


Figure 7: Extracted curve — detailed view.

Low-speed regime

- Condition: 100 < OutputSpeed < 3000 RPM and VCU_DesiredTorque > 0
- Model used: Random Forest Regressor
- Initial evaluation: high residuals, RMSE of approximately 4.568
- Solution: lag features were added to give the model temporal memory
- Result: residuals significantly reduced, RMSE improved to approximately 0.857

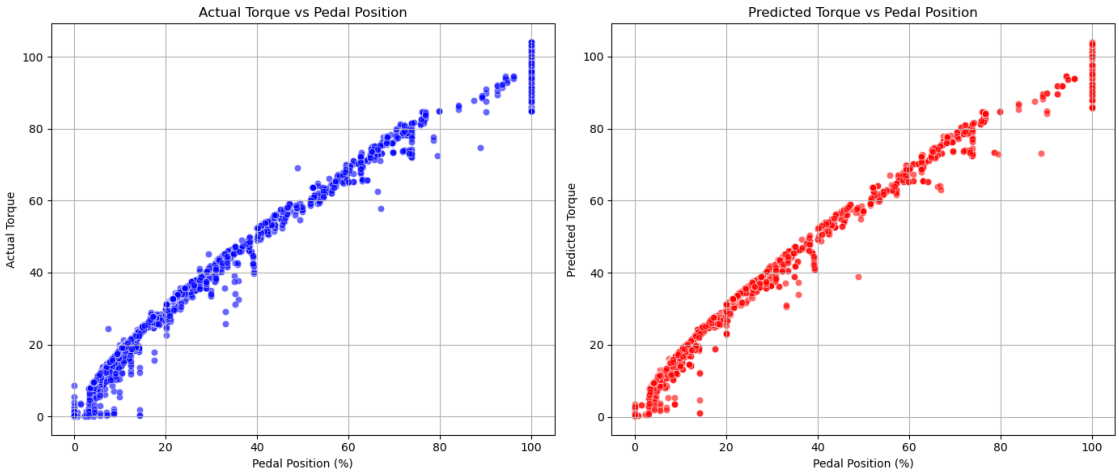


Figure 8: Low-speed regime predictions after adding lag features.

High-speed regime

- Condition: OutputSpeed > 3000 RPM and VCU_DesiredTorque > 0
- Model used: Tuned Random Forest Regressor

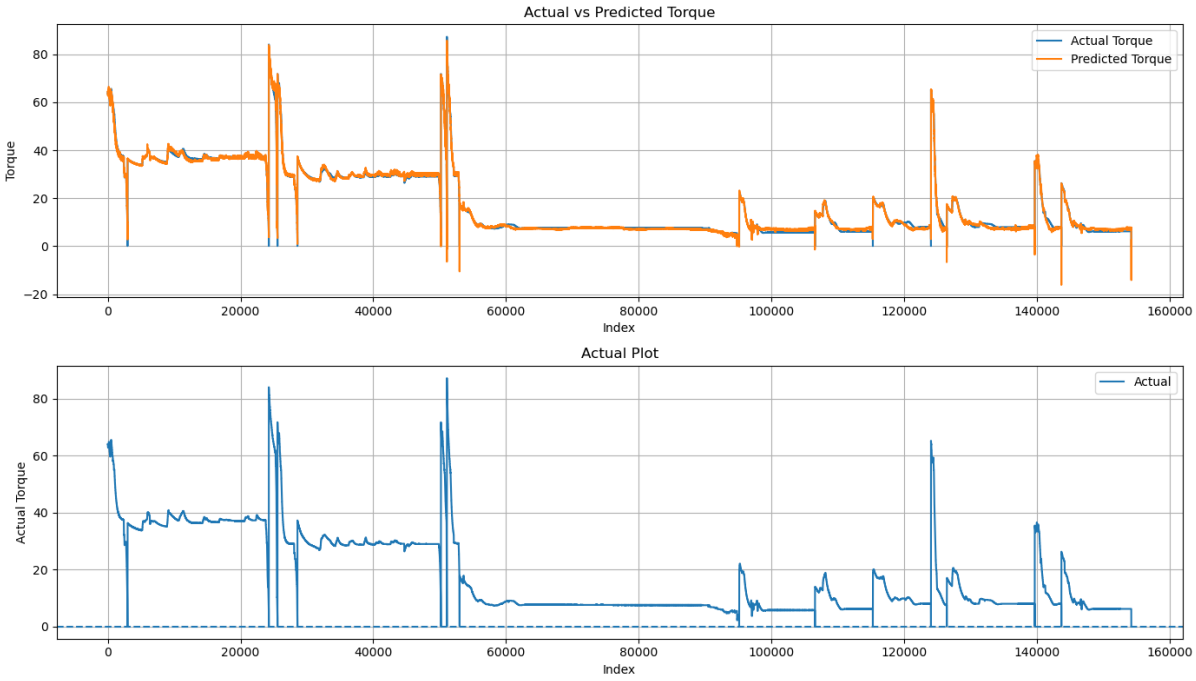


Figure 9: High-speed regime model predictions.

10.2 Eco / Drive Mode

A similar approach was used for Eco mode. However, unlike Sports mode, no distinct low-speed and high-speed pattern was observed. The model (XGBRegressor) was therefore trained on the complete torque > 0 region, while the

torque < 0 region continued to use the extracted-curve logic.

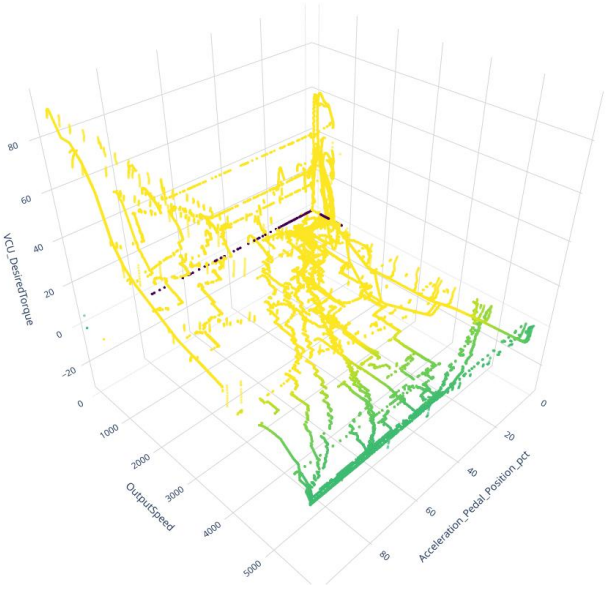


Figure 10: Eco-mode model predictions across the trained torque range.

10.3 Regen Coasting

The Regen coasting model uses an XGBoost Regressor (XGBRegressor), combined with a parametric transient “ramp” model for the moment a regen/coasting event begins.

Model Hyperparameters

Parameter	Value
n_estimators	400
max_depth	5
learning_rate	0.05
subsample	0.8
colsample_bytree	0.8
objective	squared error

Table 3: XGBoost hyperparameters used for the Regen coasting model.

Transient Ramp Model

A distinguishing feature of the Regen model is that it does not rely on the XGBoost regressor alone. A separate parametric “ramp” model (multivariate; coefficients a = -0.00337, b = 7.259, c = 0.0516) governs the torque trajectory during the first 43 samples (transient_samples = 43) after a regen event starts.

This is necessary because regen/coasting events have a distinct onset transient — when the driver lifts off the accelerator, torque needs to ramp toward the target regen value smoothly rather than jump discontinuously, which the tree-based XGBoost model (trained mostly on steady-state samples) does not capture well on its own. The samples_since_event_start feature lets the pipeline identify where in an event it currently is: for early samples within the 43-sample transient window, the parametric ramp model shapes the torque trajectory, handing off to the XGBoost regressor for the steady-state portion of the event.

(green – Predicted Torque & blue – Actual torque)



11. Post-Processing and Results Under Real Driver Behaviour

New sports data files, at the same 100 Hz sampling frequency, were used to test the Sports low/high-speed models against real driver behaviour. Performance was poor, with a much higher RMSE than previously observed.

On visualizing the data, it was found that most pedal positions were either 0 or 100, indicating more aggressive driving behaviour than the earlier driver used for training. A new pattern was also observed: while driving at high speed, the pedal remained at zero and then suddenly jumped to 100, causing the model to predict an immediate torque rise.

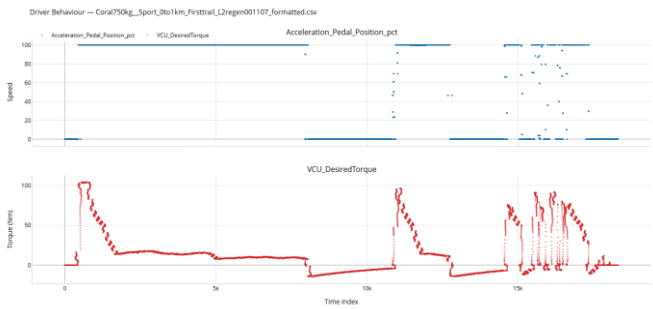


Figure 12: Sudden pedal transition from 0 to 100 at high speed, producing an abrupt predicted torque rise.

11.2 Direct Smoothing of Predicted Spikes

Instead of generating a new torque ramp using the exponential-ramp method, the predicted torque spikes were smoothed directly. The raw predicted torque was processed using an Exponential Moving Average (EMA) as a post-processing step, and gradient clipping (clipping by value) was applied to limit sudden changes in torque. This combination effectively reduced prediction spikes and produced the final output torque with a smoother transition.

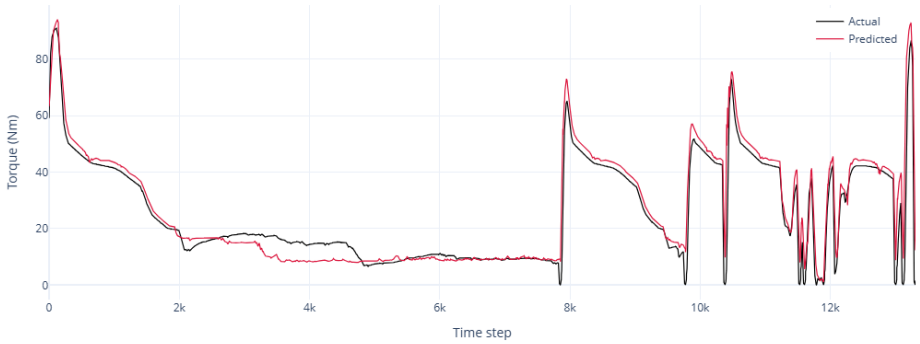


Figure 13: Final predicted torque after EMA smoothing and gradient clipping, compared to the raw spiking prediction.

11.3 Deployed Post-Processing Chain

The post-processing chain implemented in torque_predictor.c applies the following steps, in order, across modes:

Step	Operation	Notes
1	Physical clamp / clip	Restricts output to the valid torque range
2	Boost	×1.05 (×1.025 during the Regen ramp phase)
3	Median spike gate	7-sample window; Eco and Sports-low only
4	EMA smoothing	α = 0.05 steady-state; α = 0.9 during Regen transient
5	Rate limiting	5 Nm/step (Eco/Sports-low); Regen uses a combined EMA + rate-limit across the transient-to-ramp hand-off

Table 4: Deployed post-processing pipeline (torque_predictor.c).

12. Model Evaluation / Results and Analysis

12.1 Sports Mode — Low-Speed Regime

Stage	RMSE (Nm)
Initial (no lag features)	~4.568
After adding lag features	~0.857

Table 5: RMSE improvement for the Sports low-speed regime after adding lag features.

12.2 Eco Mode

The Eco-mode model produced satisfactory results on the trained torque > 0 region:

Metric	Value
MAE	1.5143 Nm
RMSE	3.0409 Nm
R^2	0.948168

Table 6: Eco-mode model evaluation metrics.

12.3 Analysis Under Real Driver Behaviour

As detailed in Section 11, testing against the sports dataset (representing a more aggressive driver) exposed a much higher RMSE than seen during initial evaluation, traced to sudden 0-to-100 pedal transitions at high speed. The adopted EMA-and-clipping post-processing approach reduced the resulting prediction spikes without altering the underlying regime models.

13. Implementation

13.1 Python Prediction Pipeline

A final Python script was developed that takes the required inputs and returns the predicted torque. The prediction logic is wrapped inside a main function, which calls the predictor() function; this function loads all three drive-mode models and follows the overall prediction workflow shown in Figure below.

13.2 Conversion to C for MATLAB Integration

To enable integration with MATLAB and support code generation, the Python implementation was converted to C. Since .pkl files are Python-specific and cannot be used directly in C, each trained model was first converted to JSON format, and the JSON files were then converted into header (.h) files, allowing the model parameters to be embedded directly into the C code. This makes the implementation self-contained, removes the dependency on Python during execution, and enables deployment in MATLAB and embedded environments.

13.3 Results in the MATLAB Block

Sports (including Regen):

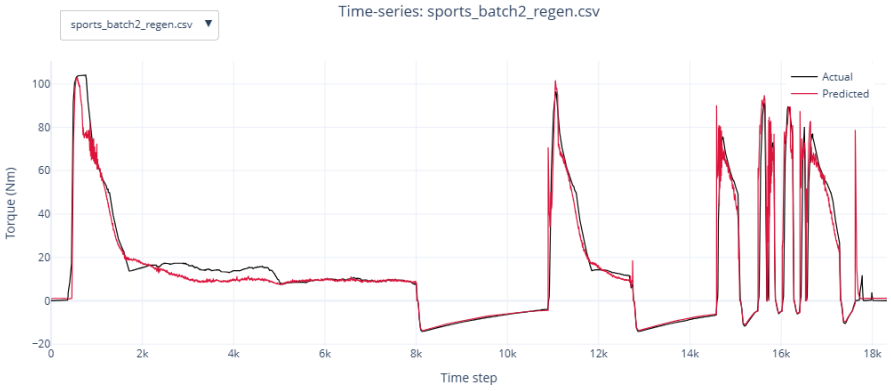


Figure 14: Predicted torque output in the MATLAB Simulink block — Sports mode including Regen.

Eco (including Regen):

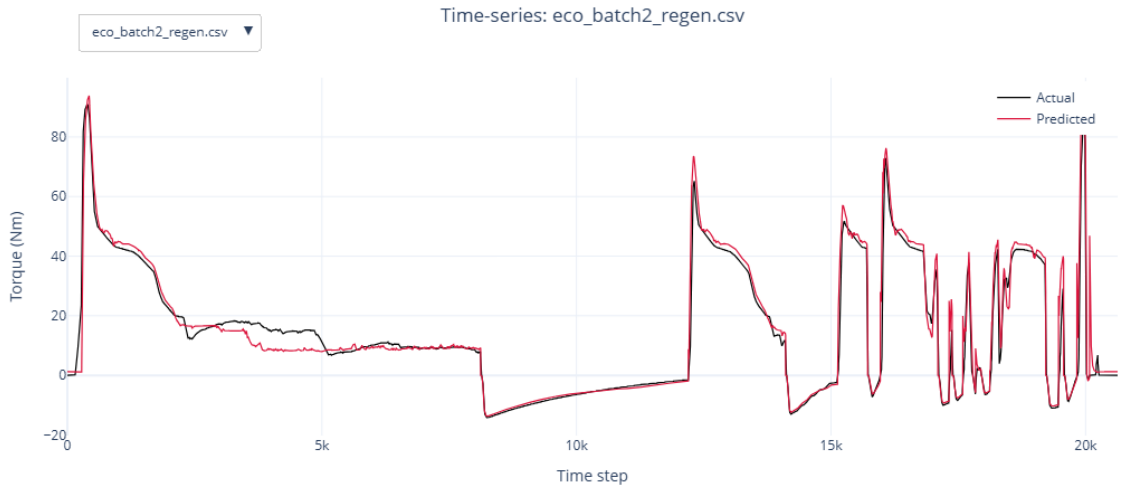


Figure 15: Predicted torque output in the MATLAB Simulink block — Eco mode including Regen.

13.4 Testing

A dedicated C test harness (test_torque_predictor.c) feeds CSV rows through the predictor, prints outputs, and performs a call-count-vs-row-count sanity check to catch call-rate mismatches against the real Simulink execution environment.

14. Challenges

- Abnormal, thread-like signal points near OutputSpeed ≈ 100 and unexpected torque values at zero pedal position, which required careful verification against related physical signals before being treated as noise and removed.
- A significant mismatch between the training driver's behaviour and a more aggressive real driver's behaviour (sports dataset), where pedal positions

were mostly at the extremes (0 or 100), causing sudden torque spikes not well represented in training data.

- The exponential torque-ramp approach, while smoothing transitions, reduced representativeness of actual vehicle behaviour and increased RMSE, requiring a different post-processing strategy.

15. Limitations

- An open diagnostic item remains for Eco-mode full-throttle predictions, which should be reviewed before any retraining work is undertaken.
- The Regen transient ramp model is a fixed parametric approximation (43-sample window with fitted coefficients) and may not generalize perfectly to all onset conditions outside the data it was fitted on.

16. Future Scope

- Evaluate the isotonic recalibration and per-RPM-bin bias correction options already available in the shared post-processing toolkit (dynamic_torque_filter.py) to address known tree-model failure modes such as leaf-value banding and under-representation of low-torque samples, should diagnostics reveal similar artifacts in the Regen model.
- Expand training data to better cover aggressive driver behaviour (rapid full pedal transitions) so that post-processing smoothing plays a smaller role in shaping the final output.
- Continue validating C-side predictions against Python reference outputs as new data batches and model versions are introduced.

17. Conclusion

This project successfully developed a regime-based, per-drive-mode torque prediction system for an electric vehicle, trained on high-frequency CAN bus data. Dividing Sports-mode data into Regen, low-speed, and high-speed regimes, and adding lag features to the low-speed model, meaningfully improved prediction The Eco-mode model achieved strong evaluation metrics, and the Regen coasting model's combination of an XGBoost regressor with a parametric transient ramp addressed the distinct onset behaviour of regen events.

The resulting pipeline — from data cleaning and regime creation through feature engineering, model training, and C/MATLAB deployment — provides a working, validated torque prediction system, along with clearly identified open items to guide future refinement.